



Software Economics, History, Theology

Paul Gray

To cite this article: Paul Gray (2012) Software Economics, History, Theology, Information Systems Management, 29:2, 161-164, DOI: [10.1080/10580530.2012.662106](https://doi.org/10.1080/10580530.2012.662106)

To link to this article: <https://doi.org/10.1080/10580530.2012.662106>



Published online: 03 Apr 2012.



Submit your article to this journal [↗](#)



Article views: 96



View related articles [↗](#)



Citing articles: 3 View citing articles [↗](#)

BOOKISMs

Software Economics, History, Theology

Paul Gray

INTRODUCTION

Don't be alarmed. Despite the title, this *BOOKISMs* does not go all social science on you. The three books reviewed are all related to software. The first is an important e-book by two information economists at MIT. They take a non-standard view of the underlying, computer-related causes and implications of the Great Recession. The second is a fascinating history of computer programming that follows the continuing and ongoing crises in software creation and the professionals who build it. The third is about creating data warehouses and business intelligence software by applying the methods (some would say theology) of agile programming.

Why We Lose Jobs Faster Than We Can Create Them

Eric Brynjolfsson and Andrew McAfee

Race Against the Machine: How the Digital Revolution Is Accelerating Innovation, Driving Productivity, and Irresistibly Transforming Employment and the Economy

Digital Frontier Press (2011) 64 pp.

Race Against the Machine is perhaps the shortest book ever reviewed in *BOOKISMs*. It is also one of the most powerful. It is written by two MIT authors Eric Brynjolfsson, a professor who specializes in information systems economics, and Andrew McAfee, the author of *Enterprise 2.0*, reviewed in the Summer, 2010 issue. They are, respectively, the director and a principal scientist of the MIT Center for Digital Business. Their basic hypothesis is that, because of the advances in digital hardware and software in recent years, we are reducing jobs at a faster rate than creating new jobs for those laid off or just joining the workforce. From the author's point of view, the underlying job problem is the result of our success. To Brynjolfsson and McAfee, the implication is that "skills and institutions have to work harder and harder to keep up lest more and more of the labor force faces technological unemployment."

The book is written from an economist's point of view. It looks at how information technologies affect jobs, skills, wages, and the economy. With the formal end of the Great Recession, the volume of layoffs declined but the number of

people hired slowed because firms acquired (mostly digital) machines. Good for the technology business, but bad for the general economy.

The authors posit three common explanations for the Great Recession—cyclic, stagnant, and "the end of work:"

- Cyclic argues that demand is inadequate.
- Stagnant assumes the cause is a decline in innovation resulting from a slowdown in developing new ideas that drive economic success.
- The third explanation is the opposite of stagnant. Based on a 1995 book "*The End of Work*" by Rifkin,¹ it argues that, because of digitization, fewer and fewer workers are required to produce the world's goods and services.

Brynjolfsson and McAfee add their voices to those of John Maynard Keynes and the Russian Nobelist, Wassily Leontief, that the third explanation is the correct one. That is, they agree with the end of work explanation but they claim not to be as pessimistic as Rifkin. Yes, many skills are now worthless and their holders are losing the race against the machine. Computers are doing many of the things (including programming and ever-improving artificial intelligence) that used to be done by people. However, the authors believe we are in the early stages of a "Great Restructuring" that will allow us to catch up in the number of jobs available by learning how to use machines as allies rather than adversaries.

A major problem in playing catch up results from Moore's law that the number of transistors in a minimum-cost integrated circuit doubles on average every 18 months. Further, Brynjolfsson and McAfee present evidence that in some domains (e.g., solving optimization problems), but not all, software speeds are also advancing at the doubling rate of Moore's law. The authors point out that computers, like steam power and electricity before them, are a general-purpose technology which interrupts and accelerates normal economic progress. The technology now infects and affects even the stodgiest industry. Although humans still beat machines in many areas, the computers are gaining. For example, acres of

people were used during the Great Depression to compute logarithms and other mathematical tables. Today such numbers are created online as needed. Similarly, in the legal profession, the discovery process for documents and e-mail is automated rather than manual, cutting costs while at the same time increasing the accuracy of the process. It is only in creative activity where humans excel. We have not yet reached the point where the machine routinely passes the Turing test.

Brynjolfsson and McAfee spend a considerable part of Chapter 3 examining the statistics that turn economists on. The economic system presumes that value creation and job creation are strongly linked; yet the Great Recession weakens and sometimes breaks this link. In brief, technologies accelerate while income stagnates. There is a “growing mismatch between rapidly advancing digital technologies and slow-changing humans.” Economists agree that the surge in productivity since the mid-1990s comes from IT. Yet in the decade of the 2000s, job growth went down by 1.1% whereas in every decade since the 1940s, jobs grew by 19.8% or more. This difference, say the authors, reflects structural changes resulting much more from plummeting hiring than massive layoffs. There is no economic law that most people will benefit from technological progress.

The authors see three causes for the mismatch: (1) more education is needed to keep up with technology, (2) more industries are becoming winner take all, which means superstars are richly rewarded and everyone else isn't, and (3) firms are spending more on capital goods (equipment and software) and less on labor. The result is real after-tax corporate profits soaring to above pre-recession levels.

So what is to be done? In Chapter 4, the authors offer an agenda for action containing 19 action items, dealing with education, entrepreneurship, investment, laws, regulations, and taxation. Unfortunately, this agenda is not up to the level of the rest of this book. Most of the agenda is mundane. Their list seems to this reviewer to be a compilation of nostrums based on various sources advocated in the popular press and coming from a variety of political agendas. For example, under education, their recommendations include invest in education, hold teachers accountable, eliminate tenure, separate instruction from testing, increase class hours, and encourage skilled immigrants to come to the United States. Under entrepreneurship, the recommendations are almost pure business school hype. They urge investment in infrastructure and more funding for basic research. In the catchall laws, regulations, and taxes section, they suggest a grab bag of ideas, some good, and some bad. For example they advocate reducing subsidies for home mortgages and financial services, patent reform, and shorter copyright protection time. What they don't explain is how their nostrums will solve the economic problems of technology overtaking employment which they brilliantly outlined in the preceding chapters.

Having established that advances in computing and in computer economics reduce the number of workers needed,

Brynjolfsson and McAfee turn to the benefits and promise of the digital frontier. They see two sources of benefits: Moore's Law, and the nature of information.

Unlike other goods, information isn't used up when consumed. If I eat a meal, you, the reader, can't eat it too; whereas if I read a book, you can also read it. We can use the information it contains to collaborate with one another to create new ideas. Furthermore, with the coming of the Internet, e-books, and e-journals, it no longer takes a year or more to publish. As a result, we are moving from an economics of scarcity to one of abundance in many areas. To the authors (and this reviewer) we are seeing a fundamental shift. It is a shift aided by the rapid growth of communications (3.8 billion mobile phones in the developing world in 2010!).

Whether you believe in a utopian digital future such as Brynjolfsson and McAfee envision or in a dystopian one of long-term misery and strife, this short, inexpensive e-book is important reading. It lays out the data and the issues of our current economic position in ways you probably did not think about previously. This slim 67 page e-book is available for \$3.99 at Amazon. It is well worth 10 times this price.

There Never Were Enough. Will There Ever Be Enough?

Nathan Ensmenger

The Computer Boys Take Over: Computers, Programmers, and the Politics of Technical Expertise

MIT Press (2010) 320 pp.

This column does not usually review books about history. *The Computer Boys Take Over* is an exception because it deals in detail with programming, a subject as vexing today as it was when the first large computers were built ~60 years ago. When computing started as a field, the conventional assumption was that the principal issue to be resolved is how to create better hardware. Coding was initially viewed as a routine task to be taken care of by the lowly paid army of women who previously hand-computed mathematical tables (see the review above of Brynjolfsson and McAfee). They “were expected to simply adapt the plans of computation already used in human computing projects to the new technology of the electronic computer.” Boy, were the people in charge surprised by the ongoing shortages and costs of much needed software skills.

Nathan Ensmenger, the author, graduated from Princeton in Civil Engineering, switched to the history of computing, taught for 10 years at the University of Pennsylvania, and is now at the University of Texas. The book is an outgrowth of his 2001 PhD dissertation. Well written, the book encapsulates the path of software from its inception through the 1980s. However, except for a brief discussion of Y2K, the book contains little about the changes in the last 20 years. Your reviewer, having lived through the period covered, found the book fascinating because it puts what is happening now into perspective.

In its early days, computer programming had the aura of a black art. Few people knew how to do it skillfully. Everyone developed their own idiosyncratic methods. Converting from an art to a science was a slow process. People who were attracted to programming were hard to find. They tended to be geeky, introverted, and scruffy, quite unlike the usual corporate types. Yet, they were in high demand. The rapid increase in the number of computers in industry and government created perceived shortages of programmers.

The crisis in demand came much more from the growth of business applications than from scientific computation. With a shortage of supply to meet the demand, salaries (and hence costs) escalated. Remember this was a time (the 1950s and 1960s) when computer science departments in colleges and universities were few, business schools did not teach information systems, and trade schools for programmers maximized their profit rather than their students' skills. As industry, with its culture of cost minimization, became involved, large firms began hiring untrained beginners and attempted to teach them in the hope of developing their own programmers. The approach was to use aptitude testing and profiles developed by the human relations department to try to find people with an innate capability. Given the poor forecasting accuracy of the methodology, access to the well-paying programming field became a matter of luck. Those who survived and proved to be good at it found it easy to change jobs. Some firms, for example, did not let their programmers go to professional meetings, fearing they would be hired away. The problem of body snatching became so severe that American Federation of Information Processing Societies banned recruiters from their meetings. The aptitude tests developed tended to favor males, leading to increase male domination of the field (see below).

The perceived shortage of programmers proved to be a long-term phenomenon. Whether it is real or not is a matter of dispute.² The argument continues as advocates and opponents of importing programmers through the H1B visa program and of outsourcing to India, China, and elsewhere battle it out on the public stage.

One of the subtexts is the role of women in programming. As indicated above, the initial assumption was that programming was trivial and could be handled by clerks, that is, mainly women, who would convert mathematical equations to instructions. The developers of the ENIAC, one of the first programmable U.S. computers believed that programming is "nothing more than an automated form of hand computation". They therefore hired the six "ENIAC girls" to be the first human programmers.³ They were chosen from among the women who supported the war effort by computing gunnery tables.

Although we are all familiar with the late Grace Hopper's work, she is an anomaly. By 1970, the percentage of women programmers was down to 30%. Recent data shows only 12% of computer science students are women.⁴ Ensmenger attributes this decline to the professionalization of the field. That is, when a group becomes professional, it wants to remove low status

people from gaining the title. Women were considered clericals and, until recent years, most did not have college degrees. Such factors as the nerd image, the perception that a college degree is needed, and the relative rarity of women assuming managerial roles during the period from 1960–1990 further contributed to the decline of women programmers.

A chapter is devoted to the many computer languages that exist. Ensmenger tells us that there were over a 1000 languages in existence by 1981. Many were developed in hope of reducing bugs and automating programming. Perversely, while they eliminated many existing bugs, they introduced new ones. Other chapters deal with professionalization through the rise of computer science; through organizations such as the Association for Computing Machinery, Data Processing Management Association, International Federation of Information Processing Societies and through software engineering with its structured programming and CASE to improve the management of software creation. The question of managing software activity runs through the book.

The conclusions chapter begins by briefly describing the Y2K problem as a lead-in to the huge maintenance burden, not from just fixing bugs but also from enhancing existing systems through additional features. Ensmenger then presents arguments pro and con on whether programming is a true or a failed profession. However, he does not take sides. Rather, he takes a middle position. Namely, he thinks about programming being a type of technician occupation much as radiological, science, engineering, or medical technicians. Like other technicians, programmers are well-educated with scientific or engineering training but who also value intuition and craft knowledge.

In summary, to Ensmenger, even though software is a six-decade success story, its "reputation and identity continue to be marked by perceptions of crisis and failure." As Frederick Brooks said: "There is no magic silver bullet."

Highly recommended.

Agile Migrates to Data Warehousing and Business Intelligence

Ken Collier

Agile Analytics: A Value-Driven Approach to Business Intelligence and Data Warehousing

Addison–Wesley (2012) 329 pp.

The agile approach to software development is a 21st century phenomenon deriving from the core principles laid out in the Agile Manifesto of 2001.⁵ The basic idea is that programmers develop and deliver working software in incremental steps, rather than going through extensive planning such as agreeing on detailed requirements before a line of code is written. It is an iterative process that focuses on achieving practical results. It has also become a doctrinaire faith for its practitioners, who

continually fight theological wars with other developers over which development approach is best.

Ken Collier, an experienced agile programmer and a long-time advocate, examines how the methodology can be used to write programs for business intelligence applications. The words “Agile Analytics” appear in the title of the book, in Chapter 1, and in the titles of each of the two parts of the book. However, the concept of analytics as conventionally understood to refer to applying quantitative methods to solving business problems, is never mentioned and the word analytics does not appear by itself in the Index.

So, what is this book really about? Collier starts out by offering a quick, summary course on agile methods. He stresses the aspects of collaboration, individuals, interactions, creating incremental pieces of working software, and responding to change which characterize the agile idea. He modifies the Agile Manifesto so it becomes:

- “Individuals and interconnections over processes and tools,
- Working data warehouse and business intelligence systems over comprehensive documentation,
- End-user and stakeholder collaboration over contract negotiation,
- Responding to change over following a plan”

Collier concedes the conventional values on the right of each statement are useful, but he values the ones on the left more. The approach reminds me of the statement a supervisor in my workplace had on his blackboard over 50 years ago “Something beats nothing all to hell.”

Following the principles of agile development, he argues for short, 1- to 3-week iterations to build usable chunks of code; value driven development that solves business problems; quality code that is fully tested and debugged; avoiding ceremonial processes (such as sign-offs); automating processes wherever possible; collaborating with users and stakeholders; and self-organizing, self-managing teams.

Following these principles is not an easy task. Few tools are available, the volume of statistics to be handled is large, a large amount of data warehouse design and build is necessary to obtain relatively basic business intelligence features, and deployment is, in effect, continual. Yet the task of building a data warehouse/business intelligence capability by conventional means is also quite hard and development projects often fail. To Collier, agile offers the opportunity to “fail fast and adapt.”

Is agile really better? Collier trots out a 2007 statistic based on a large sample which showed agile large project success rates were 72% compared to 63% for non-agile projects. That’s a 14% advantage. However, going to the same source, your reviewer observed a decrease in agile’s advantage over the years since then.

Collier concludes that agile development by itself is not enough. Agile project management is also required. Project

management should be set up so new, high value working BI features are added every two weeks. But doing good project management is still not enough. The team member’s mind-set should change so it focuses on creating a community, works closely with customers, and above all is a collaborative effort.

To the agile developer, “data plays only a supporting role.” Agile analytics people want to create features that enable users to do their job better. They use a story-driven approach. Users are urged to tell stories that “capture the essence” of the features they require in their BI systems. These stories form the basis for a conversation about what the system requirements should be. Obtaining and using stories is much more an art than a science. Collier devotes a chapter to obtaining and applying them. Obtaining and understanding stories involves close contact between user and developer. The stories are the basis for creating use-case scenarios that define story test cases and lead to story test case development, the subject of a chapter of its own. Other chapters deal with evolving excellent design and project automation.

In summary, *Agile Analytics* is about the use of interactive teams and people working together cooperatively rather than classic, formal, corporate hierarchical approaches. As you would expect, the writing contains much advocacy. Yes, the book covers most, if not all, aspects of the agile approach. If your shop uses agile methods, you will learn a few new things about how to use the methodology when faced with creating a data warehouse or a business intelligence capability. On the other hand, if you and your team are not steeped in agility, the book, by itself, is not sufficient to instantly convert your organization to the theology. As you would expect, you will need to hire and integrate IT people with a different set of skills than your organization has and to change the IT group’s relationships with your firm’s user base.

AUTHOR BIO

Paul Gray is Professor Emeritus at the School of Information Systems and Technology, Claremont Graduate University, Claremont, CA. This is his 26th year as *BOOKISM*’s editor and as a member of the editorial board of *Information Systems Management*. He is the author/editor of 15 books and author of over 160 articles. He was awarded the LEO Prize for lifetime achievement by the Association for Information Systems.

NOTES

1. Rivkin, J., *The End of Work: The Decline of the Global Labor Force and the Dawn of the Post-Market Era*. New York: Putnam Publishing Group (1995).
2. For arguments pro and con, search for “computer programmer shortage” on Google.
3. The last surviving Eniac Girl, Betty Jean Jennings, died in March 2011.
4. Source: *Wikipedia*, Retrieved January, 2012.
5. See www.agilealliance.org.